

Individual Contribution Report

CMPE 321 -- Introduction to Database Systems -- Project 3

Personal Information

Name: Faik Ihsan Sudupak

Student ID: 2022400084

Group Number: 7

Project: Modular DBMS Engine (four-layer Python implementation)

Tasks & Contributions

Our team worked on this project as a tightly coupled pair rather than splitting it into independent personal tracks. Every layer, test, and design decision was shaped through joint discussion, and my contributions are equal in scope to my teammate's, just with a different emphasis on certain components.

I focused on the File & Index Manager's record layer and the B+ tree implementation. This included the Schema dataclass, the record and catalog serialization routines (fixed-width int and null-padded string fields, alphanumeric validation, the schema-from-tokens parser), and the multi-level B+ tree with split propagation, leaf-chain range scans, and tree-header persistence. I also drove the Query Processor: command parsing, dispatch, explain plan/result/stats blocks, the three output files (output.txt, log.csv, stats_output.txt), and the failure handling that keeps the engine from crashing on bad input.

On the experiment side, I ran the index-strategy comparison (Experiment 2) and wrote the discussion contrasting hash and B+ tree on equality and range queries, including the --int-pk workload variant needed to actually exercise B+'s range fast path. I also drafted the README, the AI usage disclosure, and prepared the experiment record file.

Collaboration & Challenges

We collaborated mainly through joint working sessions and a shared Git repository, with both of us reading every change before it landed. Because the architecture is strict -- layers talk only through Result objects -- we could prototype each layer separately, then plug it in without surprises. We treated the per-layer test suites as a safety net: any change that broke an earlier suite triggered a discussion before being merged.

Two challenges stand out. The first was an ambiguity in the spec itself: section 4.3 says type and field names must be strictly alphanumeric, but the spec's own example uses field names like `military_strength` with an underscore. We discussed this and chose to accept underscores in identifiers while keeping string field values strictly alphanumeric -- and documented the choice in both DESIGN.md and the report. The second challenge was a subtle B+ tree bug where allocating an internal root and writing it back happened in two steps; isolating and fixing it taught us to treat allocate-then-write as a single atomic operation in the API surface.

Self-Assessment

This project pushed me to think carefully about disk layouts, page-level invariants, and how to make a B+ tree behave correctly under page-cache eviction. I am proud that all three index strategies share the same public surface and pass the same end-to-end tests -- the strategy pattern paid off. The skill I most improved is writing small, focused test scripts for each layer; in past projects I had relied on running the full pipeline to validate changes, but the per-layer test harnesses we built here caught regressions early and gave me confidence to refactor. Looking back, I would have spent less time tuning the hash function and more time on B+ tree delete rebalancing, which we deliberately kept simple per the spec's minimum requirements.

Use of AI Tools

We used Anthropic's Claude (Claude Opus 4.7, via claude.ai) as a design partner and code-review aid throughout the project; full details are in `ai_usage.md`. We discussed each design decision with the assistant before writing code, asked it to draft scaffolding and tests, and consulted it when debugging non-obvious failures. We never pasted output without reading and editing it, and we can explain every line of the submitted code.